

Unrolled List
–Destroy List
–Time Complexity Analysis

Program

```
void destroyList() {
    if (head == NULL) {
        cout << "The list is already empty.\n";
        return;
    }

    struct Node* temp = head;
    while (temp != NULL) {
        struct Node* nextNode = temp->next;
        free(temp);
        temp = nextNode;
    }

    head = NULL;
    cout << "The entire list has been destroyed and memory
freed.\n";
}

...
    case 5:
        destroyList();
        break;
```

Time Complexity Analysis

- ***n***: Total number of active elements.
- ***N***: Total number of node blocks ($N \approx \left\lceil \frac{n}{4} \right\rceil$).
- ***B***: Node capacity (***B*** = 4).

1. Best Case Time Complexity: $O(1)$

The absolute best – case scenario occurs when the function is called on a list that has no nodes.

- **The Condition**: The function evaluates ***if (head == NULL)***.
- **The Execution**: Because ***head*** points to ***NULL***, the condition is true. The program prints a message stating the list is empty and immediately executes a ***return;*** statement, exiting the function.
- **Result**: This single check and return process requires strictly constant time, making the Best Case **$O(1)$** .

2. Worst & Average Case Time Complexity: $O(N)$ (or $O(n)$)

If the list is populated, the ***destroyList*** function must completely wipe the data structure from memory. Because it must traverse and delete every single node to prevent memory leaks, there is no "early exit."

Thus, the worst – case and average – case times are mathematically identical.

Node Traversal (The while Loop)

```
while (temp != NULL)
```

- The function uses a `while (temp != NULL)` loop to traverse the list.
- If the list has N nodes (like the 3 like in address: `0x1000`, `0x2000`, and `0x3000`), the loop evaluates to **True** exactly N times.

```
struct Node* nextNode = temp->next;  
free(temp);  
temp = nextNode;
```

1. `struct Node * nextNode = temp → next;` (Temporarily saves the address of the next node block).

It runs : $1 + 1 + \dots N \text{ times} = 1 \times N = N$.

2. `free(temp);` (Deallocates the current node)

It runs : $1 + 1 + \dots N \text{ times} = 1 \times N = N$.

3. `temp = nextNode;` (Moves the pointer forward).

It runs : $1 + 1 + \dots N \text{ times} = 1 \times N = N$.

That is total time complexity of the loop : $N + N + N = 3N$

`free(temp)`: it frees the starting address (e.g., `0x1000`) and all of its associated memory allocations (ending at `0x1018`) all at once. Because the entire array of elements and the node metadata are destroyed simultaneously in one massive chunk, the algorithm does not need to iterate through the individual **B** elements inside.

C. Post – Loop Operations

```
    head = NULL;
    cout << "The entire list has been destroyed and memory
freed.\n";
}
```

- *Once the loop exits (when **temp** becomes **NULL**), the function sets **head = NULL**;*
- *It then prints a final confirmation message ("The entire list has been destroyed and memory freed. ").*
- *Finally, the temporary local variables (**nextNode** and **temp**) are automatically destroyed as the function ends.*
- *Cost: $O(1)$.*

$$T = \text{Total Loop Cost} + \text{Post-Loop Cost}$$

$$T = 3N + O(1)$$

Applying Big – O , we get:

$$\Rightarrow T = 3 \times O(N) + O(1)$$

$$\Rightarrow T = O(N) + O(1)$$

$$\Rightarrow T = O(N)$$

Because the total number of nodes N scales directly and linearly with the total number of elements n (where $N \approx \left\lceil \frac{n}{4} \right\rceil$), traversing N nodes takes the exact same linear time as processing n elements.

Therefore, $O(N)$ is mathematically equivalent to $O(n)$, making the final Time Complexity definitively $O(N)$ (or $O(n)$).
